

Tarea 11 - Lectura Robusta de Archivos y Limpieza de Datos

Manejo de Excepciones, I/O y Diccionarios en inventarios forestales

Jesus Enrique Flores Riera

2026-05-22

Tabla de contenidos

1	Introduccion	1
2	Ejercicio 1 - Limpiando un inventario forestal	2
2.1	Python	2
2.2	Ejercicio 1: Limpiando un inventario forestal (I/O y Excepciones)	2
2.3	R (referencia)	3
2.4	Julia (referencia)	4
3	Ejercicio 2 - Analizando datos estructurados	5
3.1	Python	5
3.2	Ejercicio 2: Analizando datos estructurados (Diccionarios y Listas)	5
3.3	R (referencia)	5
3.4	Julia (referencia)	6
4	Preguntas de Reflexion	6
4.1	1. Por que usar try-catch	6
4.2	2. Gestion de recursos: with vs do-end vs close()	7
4.3	3. La funcion de strip() y split()	7
5	Conclusiones	7

1 Introduccion

Este documento resuelve los Ejercicios 1 y 2 de la Tarea 0.11 implementando lectura robusta en Python (ejecutado), R y Julia (codigo de referencia con eval:false).

2 Ejercicio 1 - Limpiando un inventario forestal

2.1 Python

2.2 Ejercicio 1: Limpiando un inventario forestal (I/O y Excepciones)

A menudo recibimos datos de campo tomados manualmente que contienen errores tipográficos. Crea un script que realice lo siguiente:

Crea programáticamente un archivo llamado `arboles_crudos.txt` con el siguiente contenido (nota los errores intencionales en las latitudes y longitudes):

Especie,Latitud,Longitud Roble,4.6097,-74.0817 Pino,4.6XYZ,-74.0900 Cedro,4.6150,-74.0850 Eucalipto,NULA,-74.0700 Nogal,4.6200,-74.0650 Escribe una función llamada `limpiar_inventario` que abra este archivo, lea línea por línea (omitendo el encabezado) y utilice un bloque de manejo de excepciones (`try-except` / `tryCatch`) al intentar convertir las coordenadas a decimales (`float` / `numeric`).

Si la línea es válida, escríbela en un nuevo archivo llamado `arboles_limpios.csv`. Si falla, omítela pero imprime en consola una advertencia indicando qué especie tuvo el error.

```
import csv

datos_crudos = [
    ["Especie", "Latitud", "Longitud"],
    ["Roble", "4.6097", "-74.0817"],
    ["Pino", "4.6XYZ", "-74.0900"],
    ["Cedro", "4.6150", "-74.0850"],
    ["Eucalipto", "NULA", "-74.0700"],
    ["Nogal", "4.6200", "-74.0650"],
]

with open("arboles_crudos.txt", "w", encoding="utf-8") as f:
    for fila in datos_crudos:
        f.write(",".join(fila) + "\n")

def limpiar_inventario(archivo_entrada, archivo_salida):
    validos = 0
    invalidos = 0
    with open(archivo_entrada, "r", encoding="utf-8") as entrada:
        lineas = entrada.readlines()
    with open(archivo_salida, "w", encoding="utf-8", newline="") as salida:
        writer = csv.writer(salida)
        writer.writerow(["especie", "latitud", "longitud"])
        for linea in lineas[1:]:
            campos = linea.strip().split(",")
            if len(campos) != 3:
                continue
```

```

    nombre = campos[0].strip()
    lat_txt = campos[1].strip()
    lon_txt = campos[2].strip()
    try:
        lat = float(lat_txt)
        lon = float(lon_txt)
        writer.writerow([nombre, lat, lon])
        validos = validos + 1
    except ValueError:
        invalidos = invalidos + 1
        print("ADVERTENCIA:", nombre, "- lat invalida:", lat_txt)
print("Registros validos :", validos)
print("Registros omitidos:", invalidos)

limpiar_inventario("arboles_crudos.txt", "arboles_limpios.csv")

with open("arboles_limpios.csv") as f:
    print(f.read())

```

2.3 R (referencia)

```

writeLines(c(
  "Especie,Latitud,Longitud",
  "Roble,4.6097,-74.0817",
  "Pino,4.6XYZ,-74.0900",
  "Cedro,4.6150,-74.0850",
  "Eucalipto,NULA,-74.0700",
  "Nogal,4.6200,-74.0650"
), "arboles_crudos.txt")

limpiar_inventario_r <- function(entrada, salida) {
  todas <- readLines(entrada)
  out <- data.frame(especie=character(), latitud=numeric(),
                    longitud=numeric(), stringsAsFactors=FALSE)
  omitidos <- 0L
  for (linea in todas[-1]) {
    p <- strsplit(trimws(linea), ",")[[1]]
    if (length(p) != 3) next
    lat_n <- suppressWarnings(as.numeric(trimws(p[2])))
    lon_n <- suppressWarnings(as.numeric(trimws(p[3])))
    if (is.na(lat_n) || is.na(lon_n)) {
      message("ADVERTENCIA: ", trimws(p[1]))
      omitidos <- omitidos + 1L
    } else {
      out <- rbind(out, data.frame(especie=trimws(p[1]),

```

```

        longitud=lon_n, stringsAsFactors=FALSE))
    }
}
write.csv(out, salida, row.names=FALSE, quote=FALSE)
cat(sprintf("Validos: %d | Omitidos: %d", nrow(out), omitidos))
}
limpiar_inventario_r("arboles_crudos.txt", "arboles_limpios_r.csv")
print(read.csv("arboles_limpios_r.csv"))

```

2.4 Julia (referencia)

```

using Printf
lineas = ["Especie,Latitud,Longitud","Roble,4.6097,-74.0817",
          "Pino,4.6XYZ,-74.0900","Cedro,4.6150,-74.0850",
          "Eucalipto,NULA,-74.0700","Nogal,4.6200,-74.0650"]
open("arboles_crudos.txt","w") do f; write(f, join(lineas, "\n"))
end
function limpiar_inv(ent, sal)
    v=i=0
    open(sal,"w") do fo
        write(fo,"especie,latitud,longitud")
        open(ent,"r") do fi
            for ln in readlines(fi)[2:end]
                p=strip.(split(strip(ln),","))
                length(p)!=3 && continue
                try
                    lat=parse(Float64,p[2]); lon=parse(Float64,p[3])
                    write(fo,string(p[1])*","*string(lat)*","*string(lon)*"\n")
                catch; i+=1; println("ADVERTENCIA: "*string(p[1])); end
            end
        end
    end
    @printf("Validos: %d | Omitidos: %d",v,i)
end
limpiar_inv("arboles_crudos.txt","arboles_limpios.csv")

```

3 Ejercicio 2 - Analizando datos estructurados

3.1 Python

3.2 Ejercicio 2: Analizando datos estructurados (Diccionarios y Listas)

Vamos a tomar los datos limpios y pasarlos a la memoria estructurada de nuestro programa:

Crea una función `cargar_datos_espaciales` que lea el archivo `arboles_limpios.csv` que generaste en el Ejercicio 1. Por cada línea, crea un Diccionario (o Lista nombrada en R) con las llaves `especie`, `latitud` y `longitud`. Almacena todos esos diccionarios en una Lista o Arreglo principal. Recorre esa lista final y calcula programáticamente cuál es la latitud promedio de los árboles válidos, imprimiendo el resultado en consola.

```
def cargar_datos_espaciales(archivo_csv):
    registros = []
    with open(archivo_csv, "r", encoding="utf-8") as f:
        lector = csv.DictReader(f)
        for fila in lector:
            registro = {
                "especie" : fila["especie"].strip(),
                "latitud" : float(fila["latitud"]),
                "longitud" : float(fila["longitud"])
            }
            registros.append(registro)
    return registros

arboles = cargar_datos_espaciales("arboles_limpios.csv")
lat_suma = 0.0
for a in arboles:
    print(" ", a["especie"], "lat=", a["latitud"])
    lat_suma = lat_suma + a["latitud"]

lat_prom = lat_suma / len(arboles)
norte = max(arboles, key=lambda x: x["latitud"])
print("Latitud promedio :", round(lat_prom, 4))
print("Arbol mas norte :", norte["especie"])
```

3.3 R (referencia)

```
df_in <- read.csv("arboles_limpios_r.csv", stringsAsFactors=FALSE)
cargar_datos_r <- function(df) {
  lapply(seq_len(nrow(df)), function(i)
    list(especie=df[i,"especie"], latitud=df[i,"latitud"], longitud=df[i,"longitud"]))
}
arboles_r <- cargar_datos_r(df_in)
```

```

latitudes_r <- sapply(arboles_r, function(x) x[["latitud"]])
idx <- which.max(latitudes_r)
for (a in arboles_r) cat(sprintf(" %-12s lat=%.4f\n", a[["especie"]], a[["latitud"]]))
cat(sprintf("
Latitud promedio: %.4f\n", mean(latitudes_r)))
cat(sprintf("Arbol mas norte : %s\n", arboles_r[[idx]][["especie"]]))

```

3.4 Julia (referencia)

```

using Printf
function cargar_datos(archivo)
    recs=Vector{Dict{Symbol,Any}}()
    open(archivo,"r") do f
        for ln in readlines(f)[2:end]
            p=strip.(split(strip(ln),","))
            length(p)!=3 && continue
            push!(recs,Dict{Symbol,Any}(:especie=>String(p[1]),
                                         :latitud=>parse{Float64}(p[2]),:longitud=>parse{Float64}(p[3])))
        end
    end; return recs
end
arb=cargar_datos("arboles_limpios.csv")
lats=[a[:latitud] for a in arb]
for a in arb; @printf(" %-12s lat=%.4f\n",a[:especie],a[:latitud]); end
@printf("Latitud promedio: %.4f\n",sum(lats)/length(lats))

```

4 Preguntas de Reflexion

4.1 1. Por que usar try-catch

Sin manejo de excepciones, un error en el registro 499999 de un archivo con 500000 filas detiene todo el programa y pierde el trabajo previo. Con try-except y tryCatch el error se captura, se imprime una advertencia y el ciclo continua.

```

datos = ["4.6097", "4.6XYZ", "4.6150", "NULA", "4.6200"]

print("SIN manejo de excepciones:")
try:
    for v in datos:
        numero = float(v)
        print(" OK:", numero)
except ValueError as err:
    print(" ERROR FATAL:", err)
    print(" Valores restantes NO procesados.")

print("CON manejo de excepciones:")
for v in datos:
    try:
        numero = float(v)
        print(" OK:", round(numero, 4))
    except ValueError:
        print(" ADVERTENCIA:", v, "invalido. Continuando...")

```

4.2 2. Gestion de recursos: with vs do-end vs close()

Dejar archivos abiertos agota los descriptores del sistema operativo, corrompe datos en el buffer y genera bloqueos en redes NFS. Python usa with open para cierre automatico. Julia usa open do end. R requiere on.exit(close(con), add=TRUE).

4.3 3. La funcion de strip() y split()

Los archivos de campo tienen caracteres invisibles como saltos de linea de Windows que hacen fallar float() sin ninguna pista visible. strip() y trimws() los eliminan antes de la conversion. split() y strsplit() dividen cada linea en sus campos correctos.

5 Conclusiones

1. El manejo de excepciones garantiza que un dato corrupto no destruya miles de registros validos.
2. Los patrones with y do-end son mas seguros que close manual.
3. strip() y split() son la primera linea de defensa contra errores invisibles en datos de campo.
4. Los diccionarios hacen el codigo autoexplicativo y resistente a cambios en el esquema del CSV.